

Multi-Node Edge Processing System Architecture

Requirements Specification, Database Schema & Execution Plan

System State:	Multi-Node Refactor Specification	Document Date:	August 2026
Target Architecture:	Flask + PyTorch + SQLite + ESP32	Status:	Approved for Implementation

1. Executive Summary & Architecture Blueprint

This technical specification outlines the design and step-by-step roadmap for upgrading the edge monitoring infrastructure into a scalable multi-node processing system. The architecture isolates hardware nodes using dynamic identification, decoupling high-throughput camera frame ingestion from PyTorch model inference using an asynchronous job queue.



2. Technical Requirements Specification

- **Node Identity & Discovery:** Every sensor and camera node transmits a unique identifier (`node_id`) in JSON payloads, HTTP headers, and registration pings.

- **Asynchronous Detection Queue:** Image uploads are accepted instantly (HTTP 202) and enqueued for non-blocking execution through the PyTorch `WaveletEnhancedResNet` inference worker.
- **Camera Proxy Layer:** Backend provides proxy API routes to trigger or capture single image frames from targeted camera nodes using target IP resolution.
- **Data Isolation & Rolling Anomaly Engine:** Sensor data and image logs are indexed per node. Continuous telemetry is analyzed via rolling Z-score evaluation ($Z = |x - \mu| / \sigma > 2.5$) to flag local node anomalies.
- **Thread-Safe SQLite Persistence:** SQLite configured with Write-Ahead Logging (`PRAGMA journal_mode=WAL;`) and centralized system logging to record operating events and exceptions.

3. Database Schema (Multi-Node SQLite DDL)

```
-- Node Registry Table
CREATE TABLE IF NOT EXISTS nodes (
    node_id TEXT PRIMARY KEY,
    node_type TEXT NOT NULL CHECK(node_type IN ('sensor', 'camera', 'hybrid')),
    ip_address TEXT NOT NULL,
    mac_address TEXT,
    last_seen DATETIME DEFAULT CURRENT_TIMESTAMP,
    status TEXT DEFAULT 'active' CHECK(status IN ('active', 'inactive', 'degraded'))
);

-- Sensor Telemetry Logs (Indexed by Node)
CREATE TABLE IF NOT EXISTS sensor_logs (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    node_id TEXT NOT NULL,
    timestamp DATETIME DEFAULT CURRENT_TIMESTAMP,
    pir INTEGER NOT NULL,
    sound INTEGER NOT NULL,
    gas INTEGER NOT NULL,
    distance_cm REAL NOT NULL,
    humidity REAL NOT NULL,
    temp_c REAL NOT NULL,
    fusion_score REAL NOT NULL,
    is_anomaly INTEGER DEFAULT 0,
    FOREIGN KEY(node_id) REFERENCES nodes(node_id)
);

-- Image Captures & Model Predictions
CREATE TABLE IF NOT EXISTS capture_logs (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    node_id TEXT NOT NULL,
    timestamp DATETIME DEFAULT CURRENT_TIMESTAMP,
    file_path TEXT NOT NULL,
    trigger_source TEXT CHECK(trigger_source IN ('hardware_node', 'manual_dashboard', 'backend_api')),
    prediction INTEGER,
    sensor_log_id INTEGER,
    processing_status TEXT DEFAULT 'pending' CHECK(processing_status IN ('pending', 'processing', 'completed', 'failed')),
    FOREIGN KEY(node_id) REFERENCES nodes(node_id),
    FOREIGN KEY(sensor_log_id) REFERENCES sensor_logs(id)
);

-- System Event Logs
CREATE TABLE IF NOT EXISTS system_logs (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    timestamp DATETIME DEFAULT CURRENT_TIMESTAMP,
    level TEXT NOT NULL CHECK(level IN ('DEBUG', 'INFO', 'WARNING', 'ERROR', 'CRITICAL')),
    node_id TEXT,
    module TEXT NOT NULL,
    message TEXT NOT NULL,
    details_json TEXT,
    FOREIGN KEY(node_id) REFERENCES nodes(node_id)
);

-- Index Optimizations
```

```
CREATE INDEX IF NOT EXISTS idx_sensor_node_time ON sensor_logs(node_id, timestamp);
CREATE INDEX IF NOT EXISTS idx_capture_node_time ON capture_logs(node_id, timestamp);
CREATE INDEX IF NOT EXISTS idx_system_level_time ON system_logs(level, timestamp);
```

4. REST API Specifications

Route	Method	Headers / Parameters	Description
<code>/api/v1/nodes/register</code>	POST	Body: <code>{"node_id": "cam_01", "ip_address": "10.30.50.213"}</code>	Registers node dynamic IP mapping.
<code>/api/v1/upload</code>	POST	X-Node-ID: <code>cam_01</code>	Ingests JPEG frame into queue; returns 202.
<code>/api/v1/camera/trigger</code>	POST	Query: <code>node_id=cam_01</code>	Proxies HTTP request to camera node <code>trigger</code> .
<code>/api/v1/camera/capture</code>	GET	Query: <code>node_id=cam_01</code>	Fetches raw JPEG from camera <code>capture</code> .
<code>/api/v1/sensors/history</code>	GET	Query: <code>node_id</code> , <code>start</code> , <code>end</code>	Queries historical telemetry and anomaly markers.
<code>/api/v1/logs</code>	GET	Query: <code>level</code> , <code>node_id</code> , <code>limit</code>	Queries structured system logs.

5. Implementation Roadmap

Phase 1: Storage Infrastructure & Node Registry

- Implement thread-safe database connector (`database.py`) enabling WAL mode.
- Build `node_manager.py` for registry persistence and node keep-alive checks.
- Configure structured logger persisting events directly into `system_logs`.

Phase 2: Asynchronous Task Queue & Model Worker

- Build thread worker queue (`concurrent.futures.ThreadPoolExecutor`).
- Refactor `/upload` endpoint to return HTTP 202 immediately upon receiving JPEG payload.
- Worker thread runs `WaveletEnhancedResNet` PyTorch model inference and logs results to `capture_logs`.

Phase 3: Sensor Poller & Anomaly Detection Pipeline

- Build background poller thread executing periodic HTTP GET requests to registered sensor nodes.
- Calculate rolling baseline statistical parameters per node for anomaly determination ($Z > 2.5$).

Phase 4: Web Dashboard Interface

- Implement interactive dashboard UI featuring a node target dropdown selector.
- Render time-series telemetry charts with highlighted anomaly overlays and real-time metric streams.
- Provide manual camera trigger control panel and system event log browser.

6. Risk Matrix & Mitigations

Risk Event	Severity	Mitigation Strategy
Database Lock Contention	High	Enable WAL mode (<code>PRAGMA journal_mode=WAL;</code>) and set busy timeout to 10.0s.
Inference Queue Backlog	High	Enforce maximum queue depth; discard old items if queue breaches limit and log warning.
Dynamic IP Drift	Medium	Nodes submit registration pings on boot and periodic keep-alive heartbeats every 60s.
Camera Unresponsiveness	Medium	Set 3.0s strict request timeouts on all outgoing backend proxy requests to camera nodes.